

Natural Language Processing

CSCI 5832—Lecture 11

Jim Martin

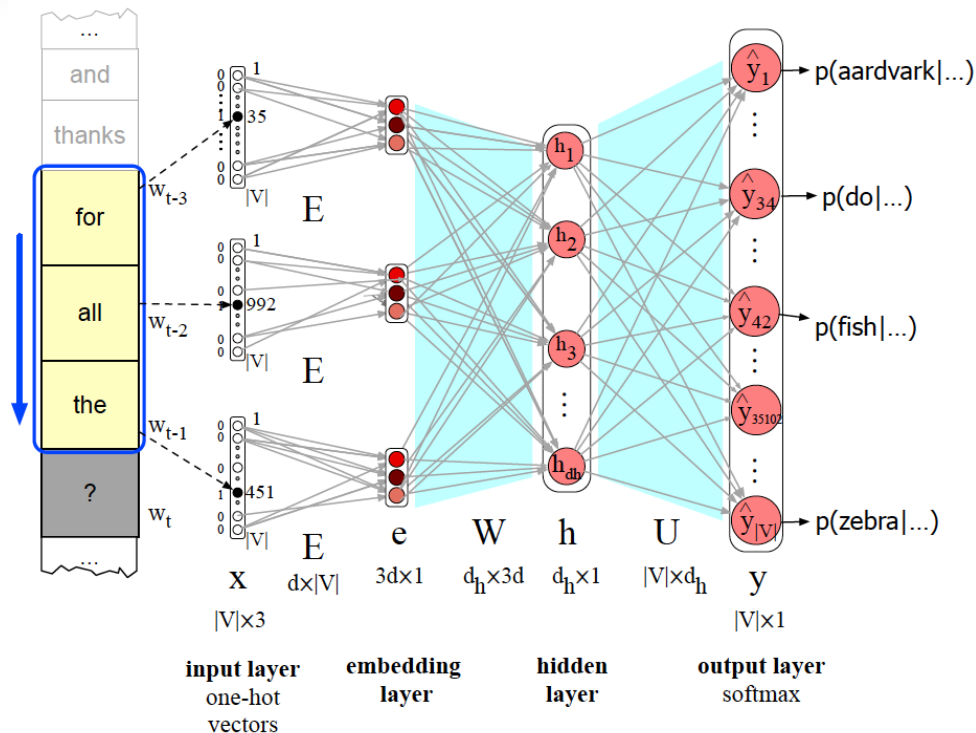
Today

- We're regrading the exams. Expect that some scores will change.
- Wrap up RNNs
- Start on Transformers
 - Self-attention

But

That sliding window is annoying. It brings back that Markov assumption.

- We'd like to take the entire context back to the beginning into account.

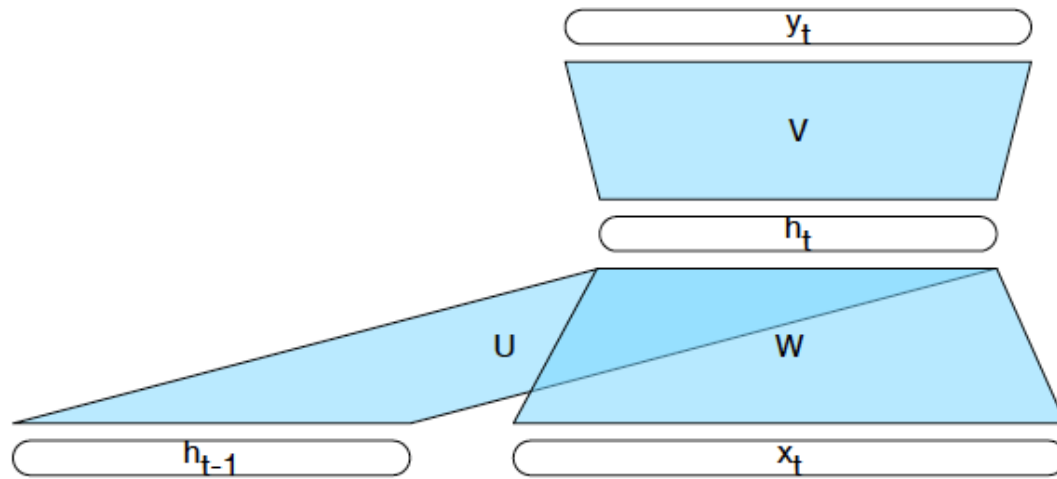
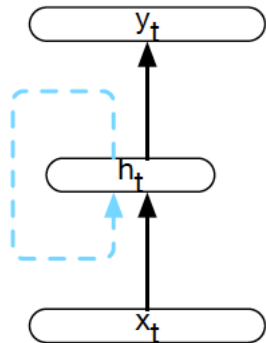


$$P(w_{1:n}) = \prod_{i=1}^n P(w_i | w_{<i})$$

Recurrent Neural Network

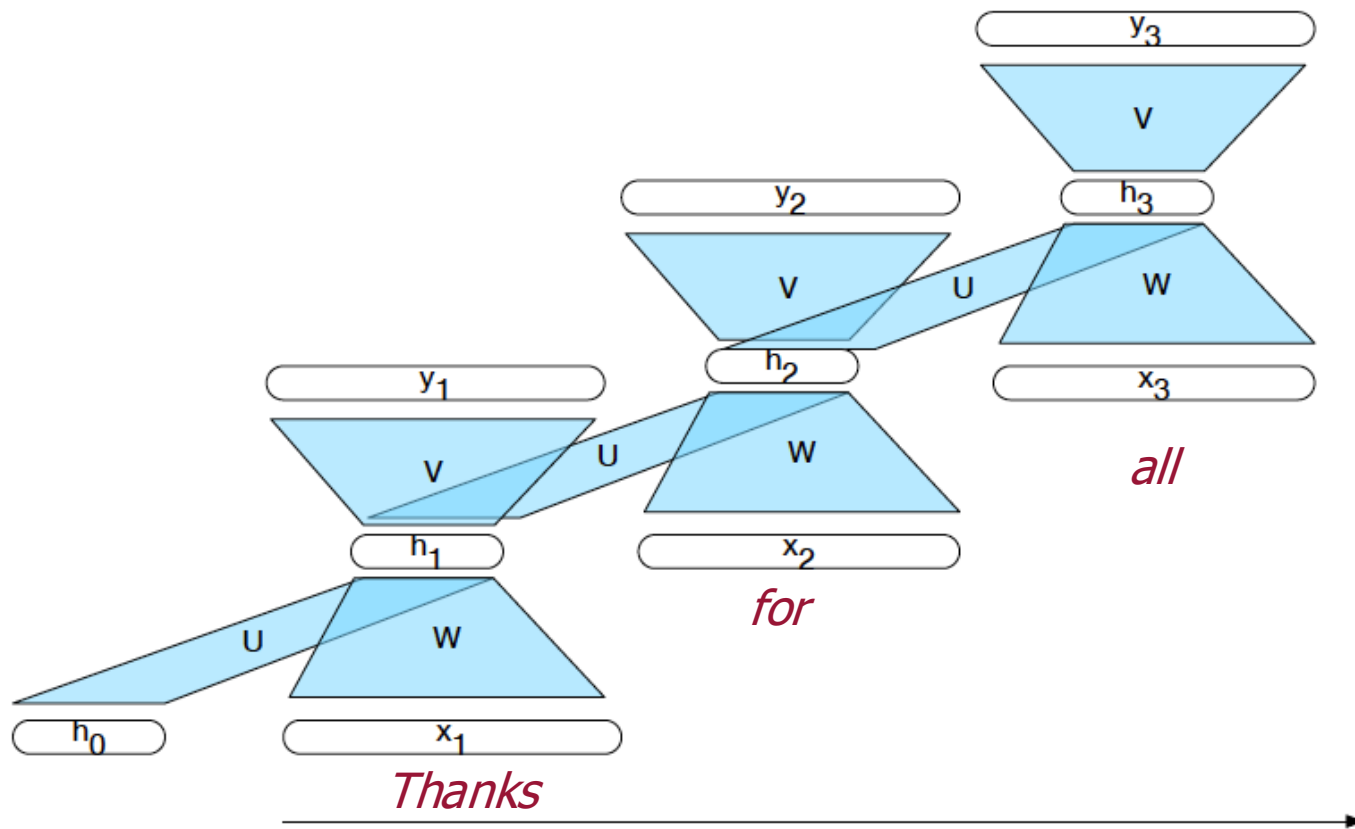
Feed words/input tokens in a sequence one at a time, compute a corresponding output at each step

- Add a copy of the previous hidden layer as input to the current hidden to provide access to earlier processing



Thanks

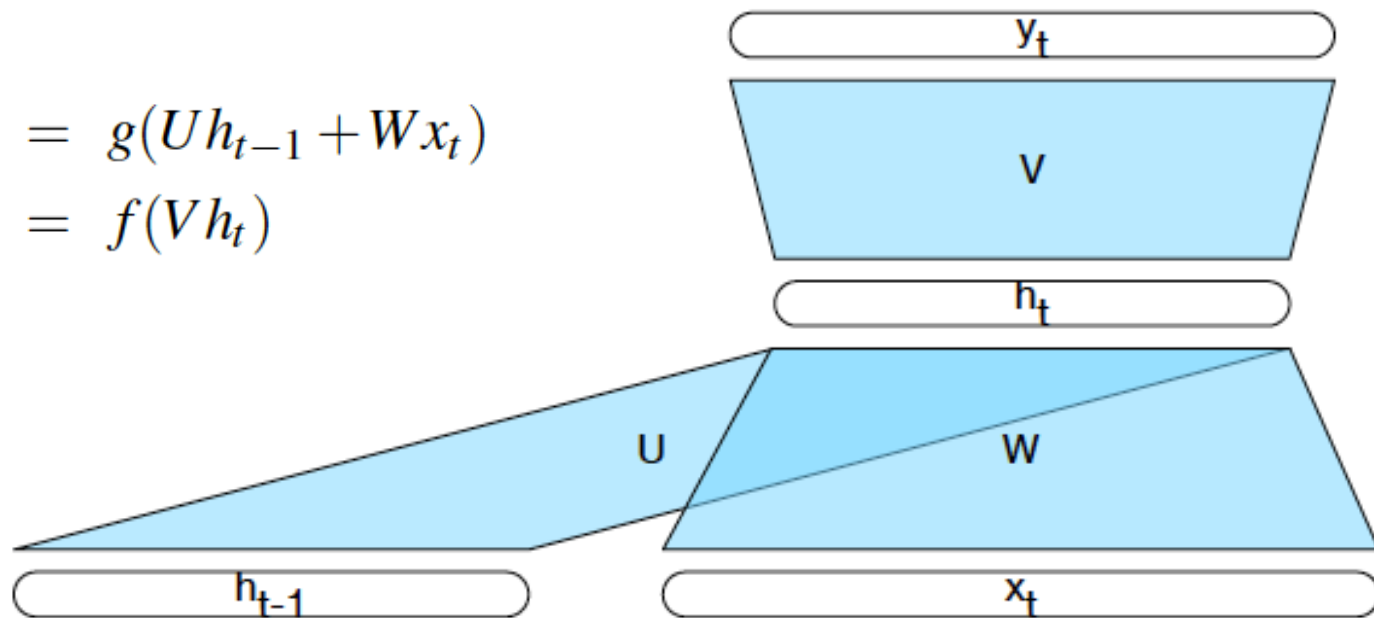
Unrolled in Time



Inference in RNNs

$$h_t = g(Uh_{t-1} + Wx_t)$$

$$y_t = f(Vh_t)$$



Sequence Processing

function FORWARDRNN($x, network$) **returns** output sequence y

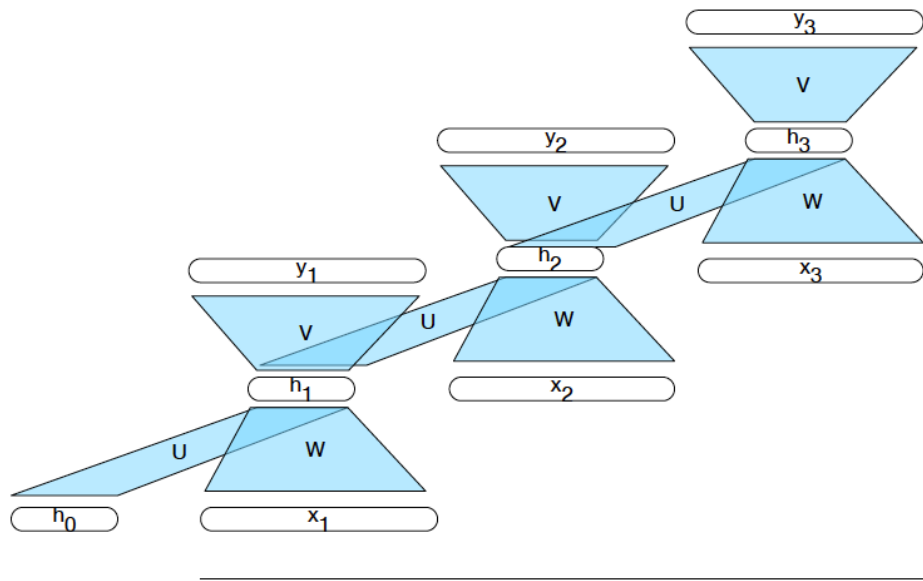
$h_0 \leftarrow 0$

for $i \leftarrow 1$ **to** LENGTH(x) **do**

$h_i \leftarrow g(U h_{i-1} + W x_i)$

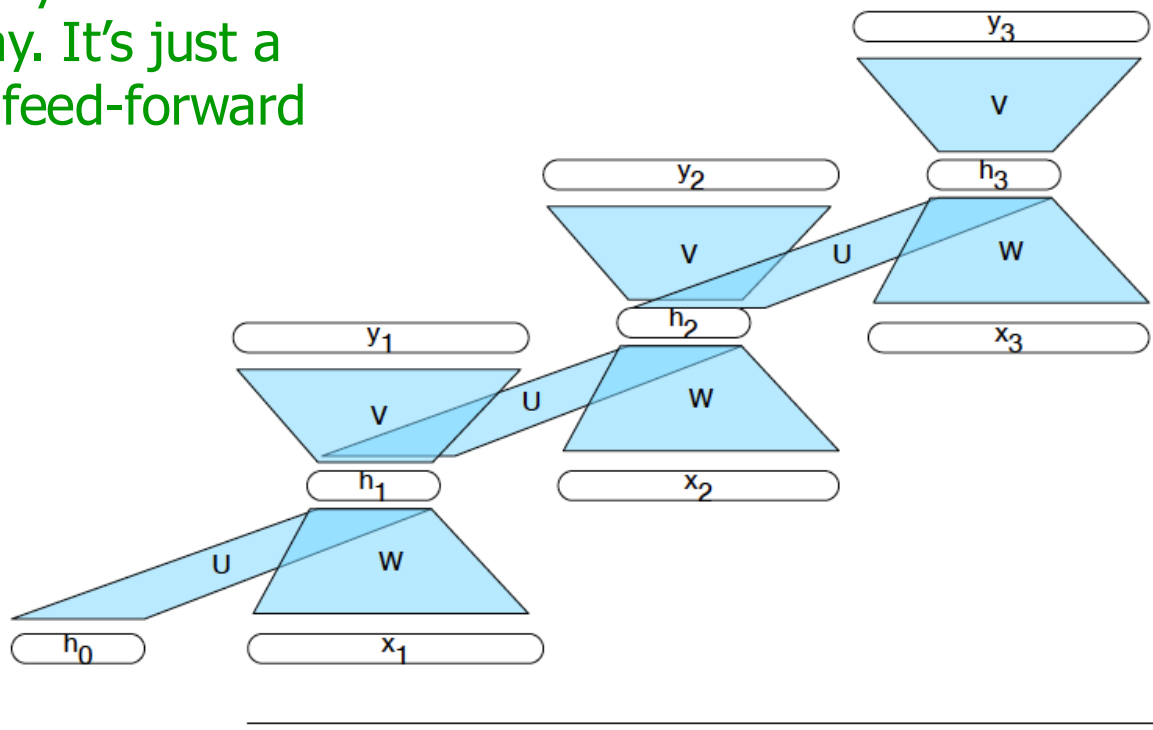
$y_i \leftarrow f(V h_i)$

return y



Computation Graphs and Unrolling

There really isn't any recurrence if you view it this way. It's just a more complicated feed-forward network.

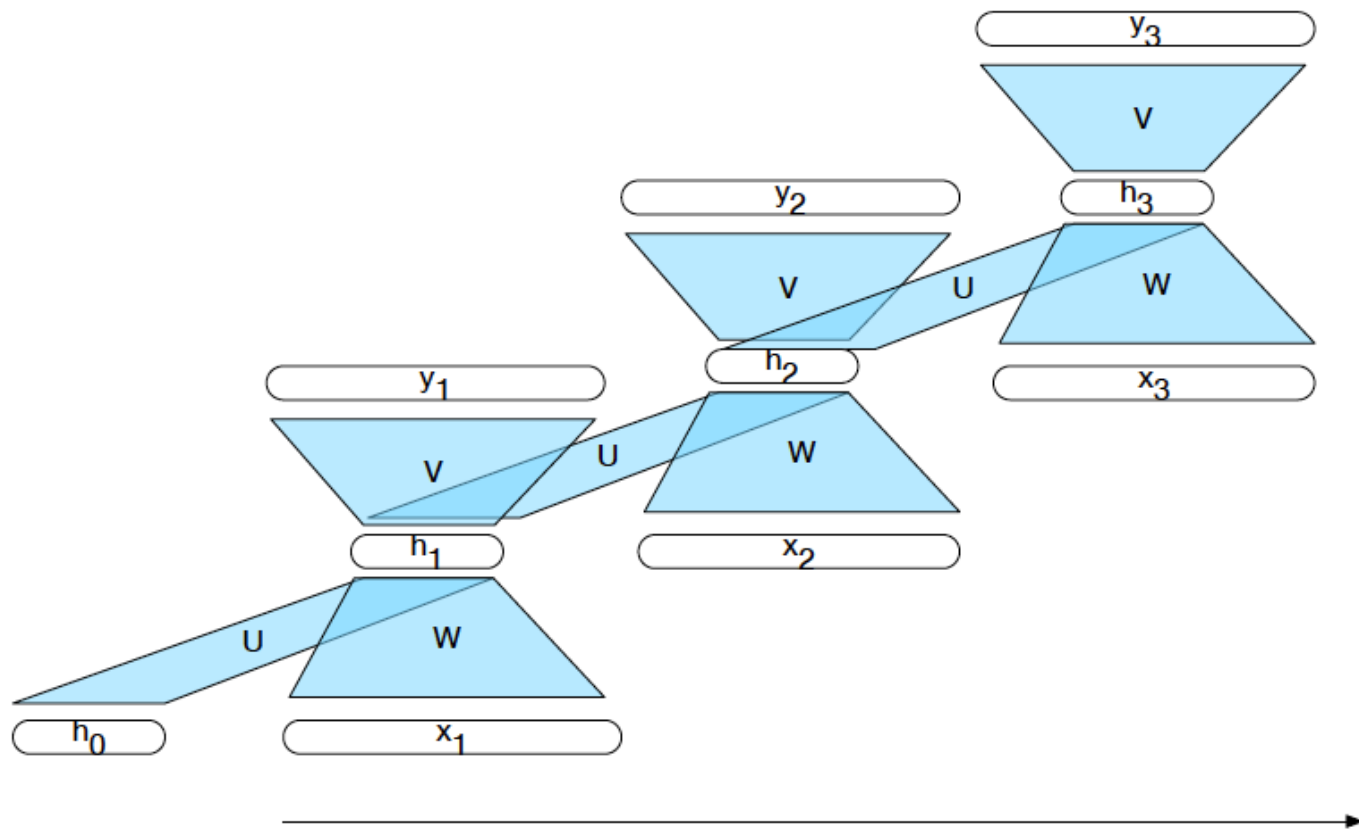


Learning

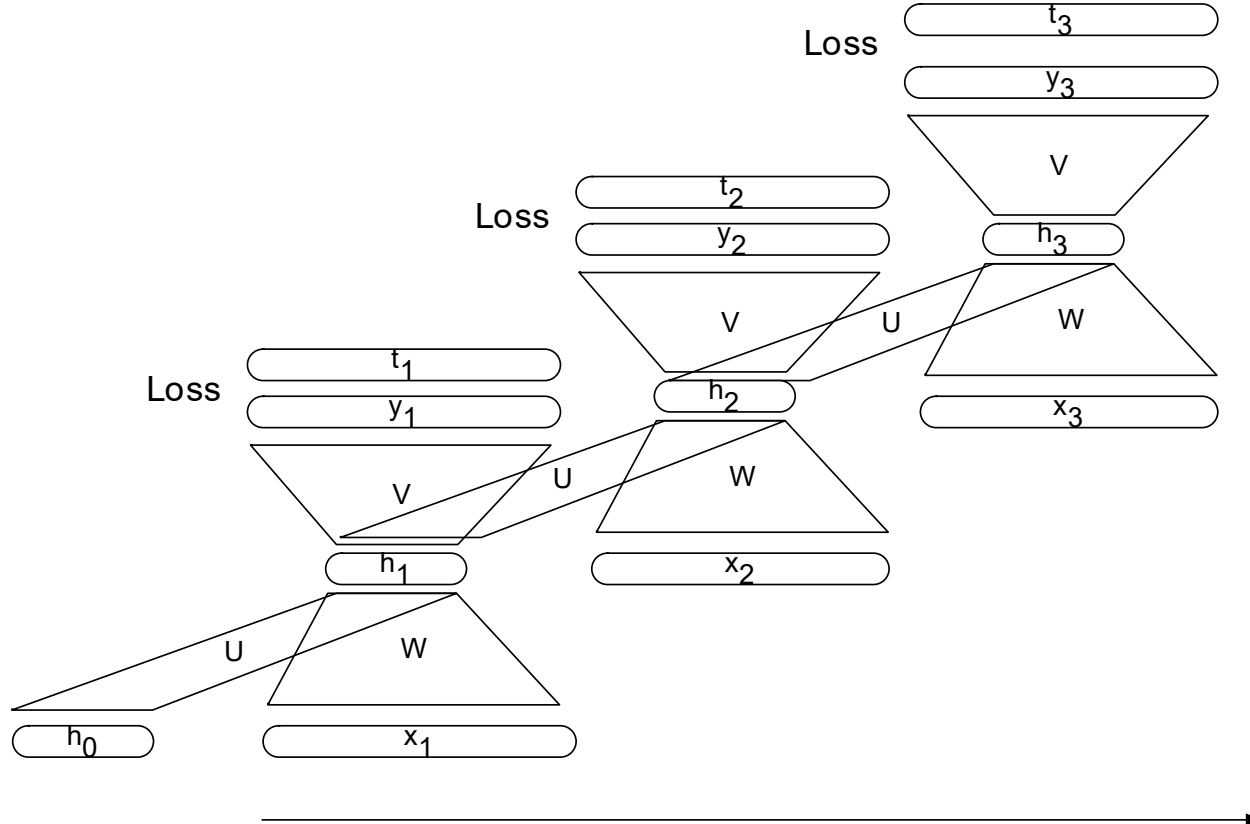
Proceeds pretty much as with normal feedforward networks

- Need training data
- Weights are randomly initialized
- Weight updates are based on gradients computed from the computation graph
- Weights are shared across timesteps

RNN Unrolled in Time



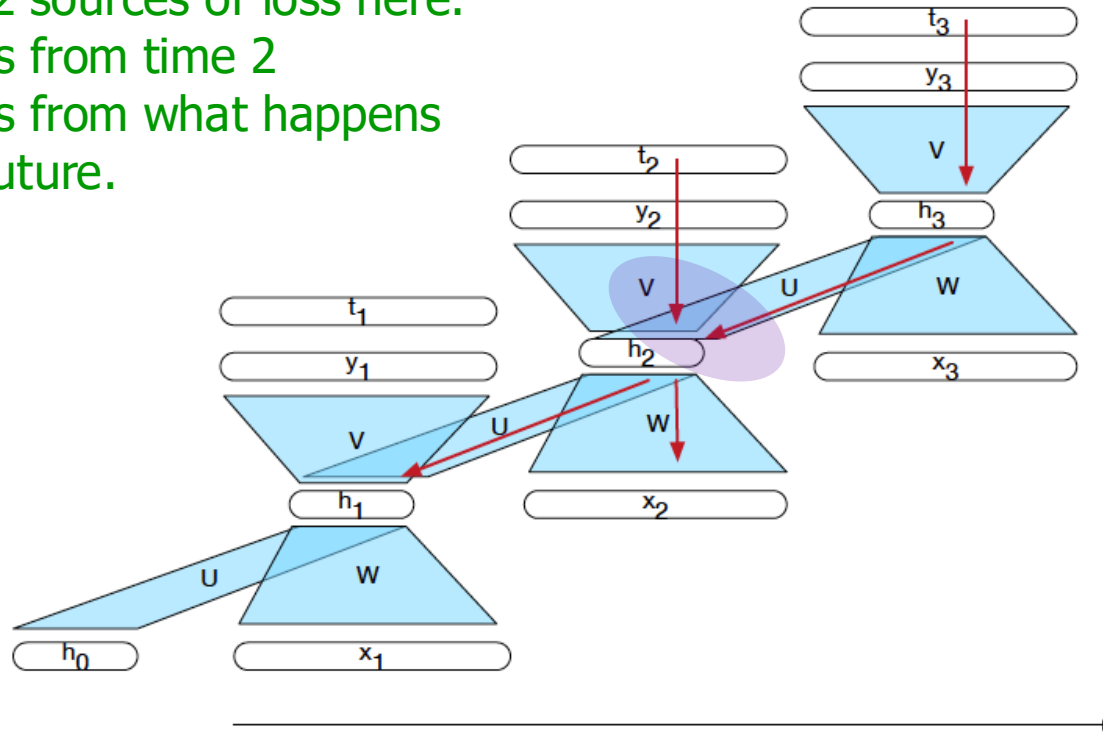
RNN: Forward Pass



RNN : Backward Pass

There are 2 sources of loss here.

1. The loss from time 2
2. The loss from what happens in the future.



RNN LM Parameter Review

Assume

- Vocab of size 50,000.
- Word embeddings length 300
- Hidden layer is length 1000

$$\mathbf{e}_t = \mathbf{E}\mathbf{x}_t$$

$$\mathbf{h}_t = g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{e}_t)$$

$$\mathbf{y}_t = \text{softmax}(\mathbf{V}\mathbf{h}_t)$$

Therefore

- $\mathbf{x}_t$ is 50,000 x 1 1-hot vector
- $\mathbf{e}_t$ is 300 x 1
- $\mathbf{h}_t$ is 1000 x 1
- $\mathbf{y}_t$ is 50,000 x 1

RNN LM Parameters

■ Giving

- $\mathbf{x}_t$ is 50000 x 1 1-hot vector
- $\mathbf{e}_t$ is 300 x 1
- $\mathbf{h}_t$ is 1000 x 1
- $\mathbf{y}_t$ is 50000 x 1

■ Therefore

- $\mathbf{E}$ is 300 x 50,000
- $\mathbf{W}$ is
- $\mathbf{U}$ is
- $\mathbf{V}$ is

$$\mathbf{e}_t = \mathbf{E}\mathbf{x}_t$$

$$\mathbf{h}_t = g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{e}_t)$$

$$\mathbf{y}_t = \text{softmax}(\mathbf{V}\mathbf{h}_t)$$

RNN LM Parameters

■ Giving

- $\mathbf{x}_t$ is 50000 x 1 1-hot vector
- $\mathbf{e}_t$ is 300 x 1
- $\mathbf{h}_t$ is 1000 x 1
- $\mathbf{y}_t$ is 50000 x 1

■ Therefore

- $\mathbf{E}$ is 300 x 50,000
- $\mathbf{W}$ is 1000 x 300
- $\mathbf{U}$ is 1000 x 1000
- $\mathbf{V}$ is 50,000 x 1000

$$\mathbf{e}_t = \mathbf{E}\mathbf{x}_t$$

$$\mathbf{h}_t = g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{e}_t)$$

$$\mathbf{y}_t = \text{softmax}(\mathbf{V}\mathbf{h}_t)$$

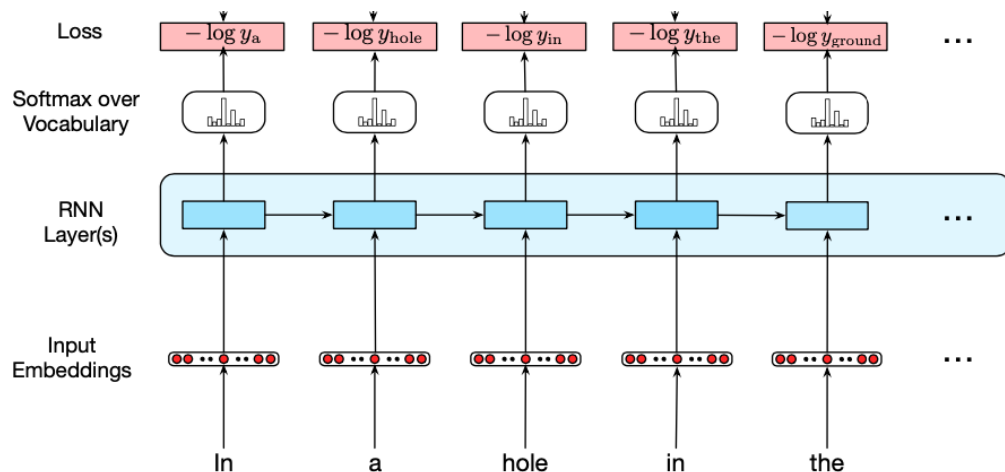
RNN LM Parameters

■ Sizes

- E is 300×50000 15M
 - W is 1000×300 300k
 - U is 1000×1000 1M
 - V is 50000×1000 50M
- 66.3M parameters

That's for 1 RNN layer. 10 layers would be?

RNN as Language Model

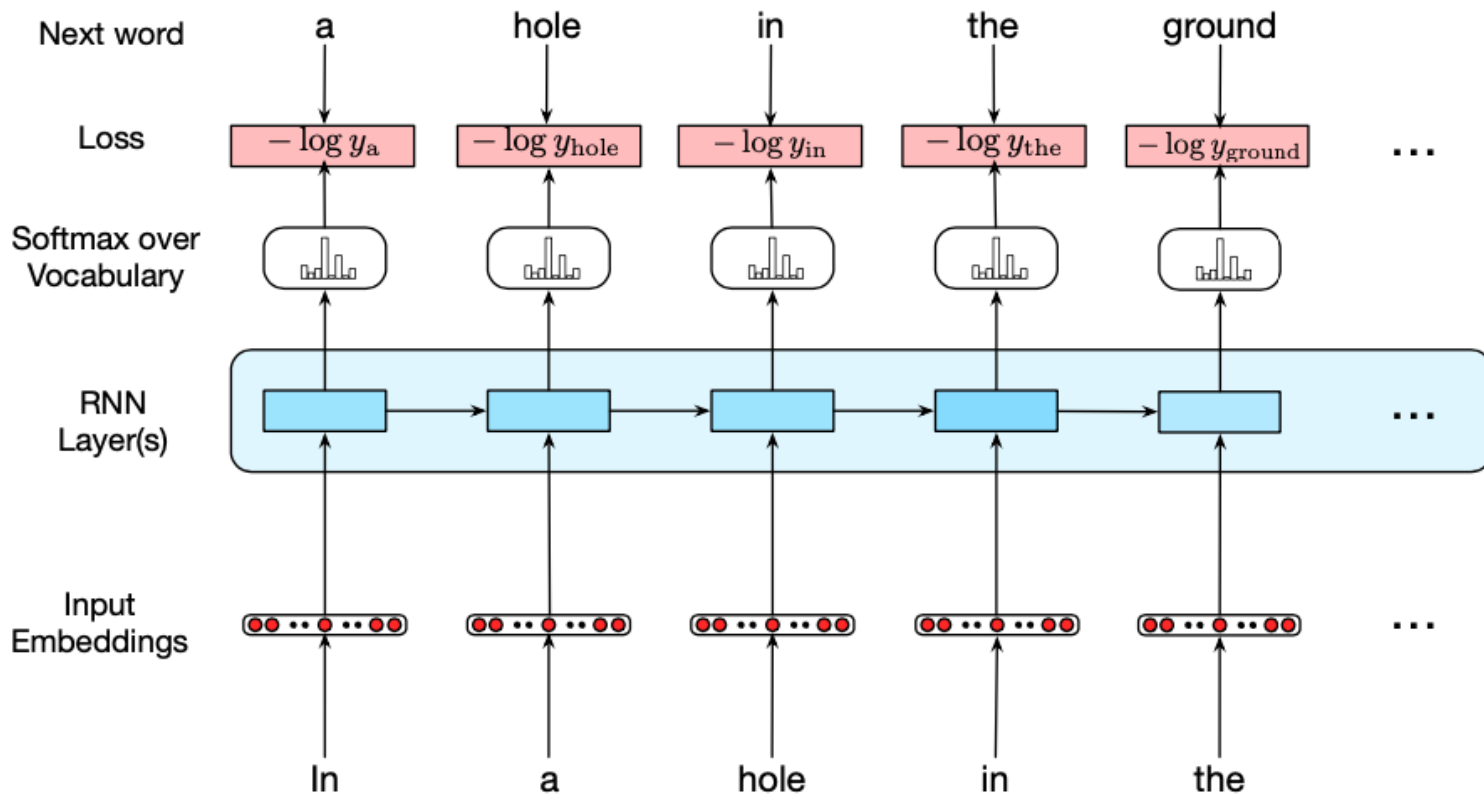


$$\mathbf{e}_t = \mathbf{E}\mathbf{x}_t$$

$$\mathbf{h}_t = g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{e}_t)$$

$$\mathbf{y}_t = \text{softmax}(\mathbf{V}\mathbf{h}_t)$$

Training an RNN Language Model



RNN Language Models

- Once trained, we have the word embeddings and the E, U, V and W matrices we need to compute sequence probabilities.

$$P(w_{t+1} = i | w_{1:t}) = y_t^i \quad P(w_{1:n}) = \prod_{i=1}^n P(w_i | w_{1:i-1})$$

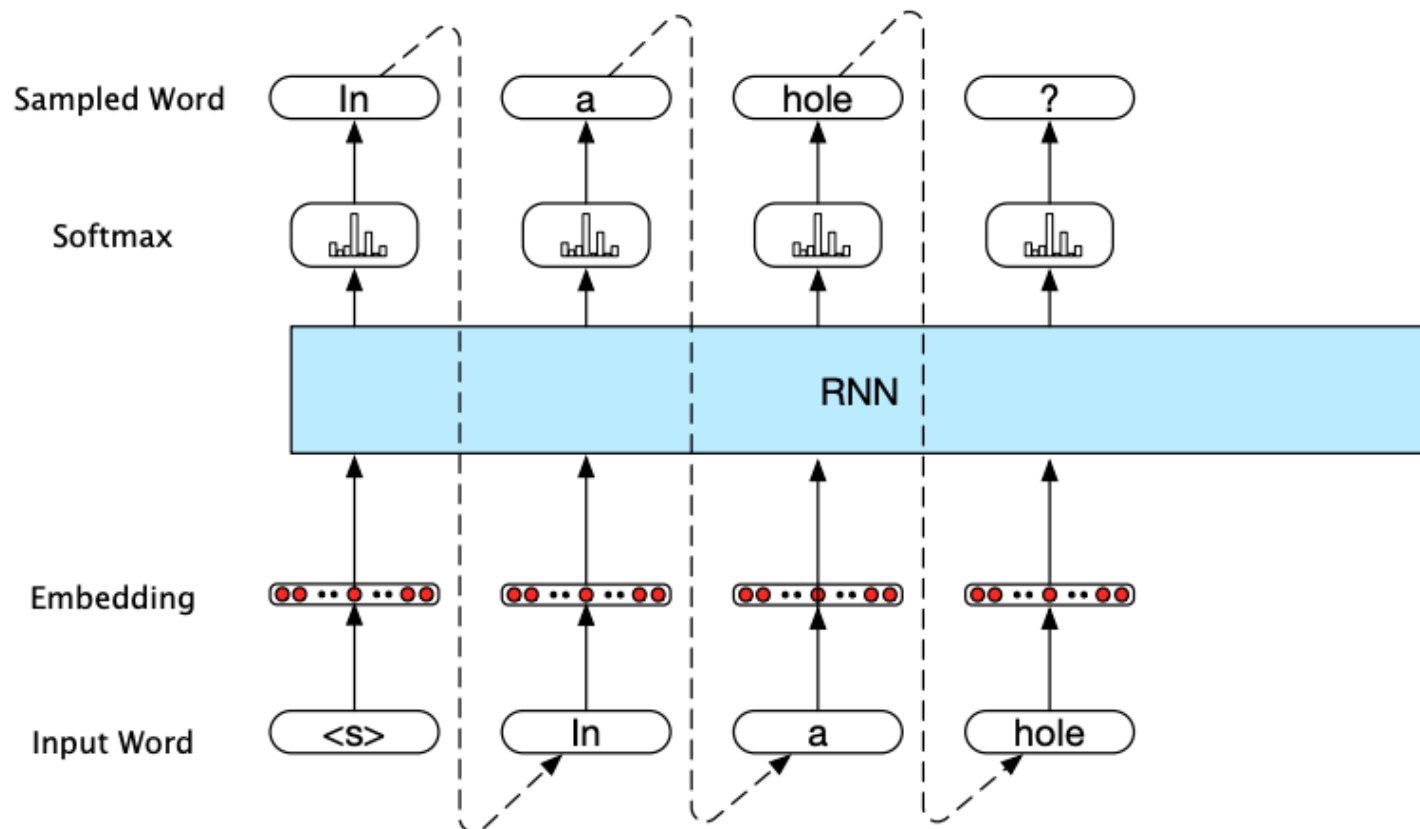
The probability that the word at time t+1 is some particular word i is just the ith component of y_t

$$= \prod_{i=1}^n y_{w_i}^i$$

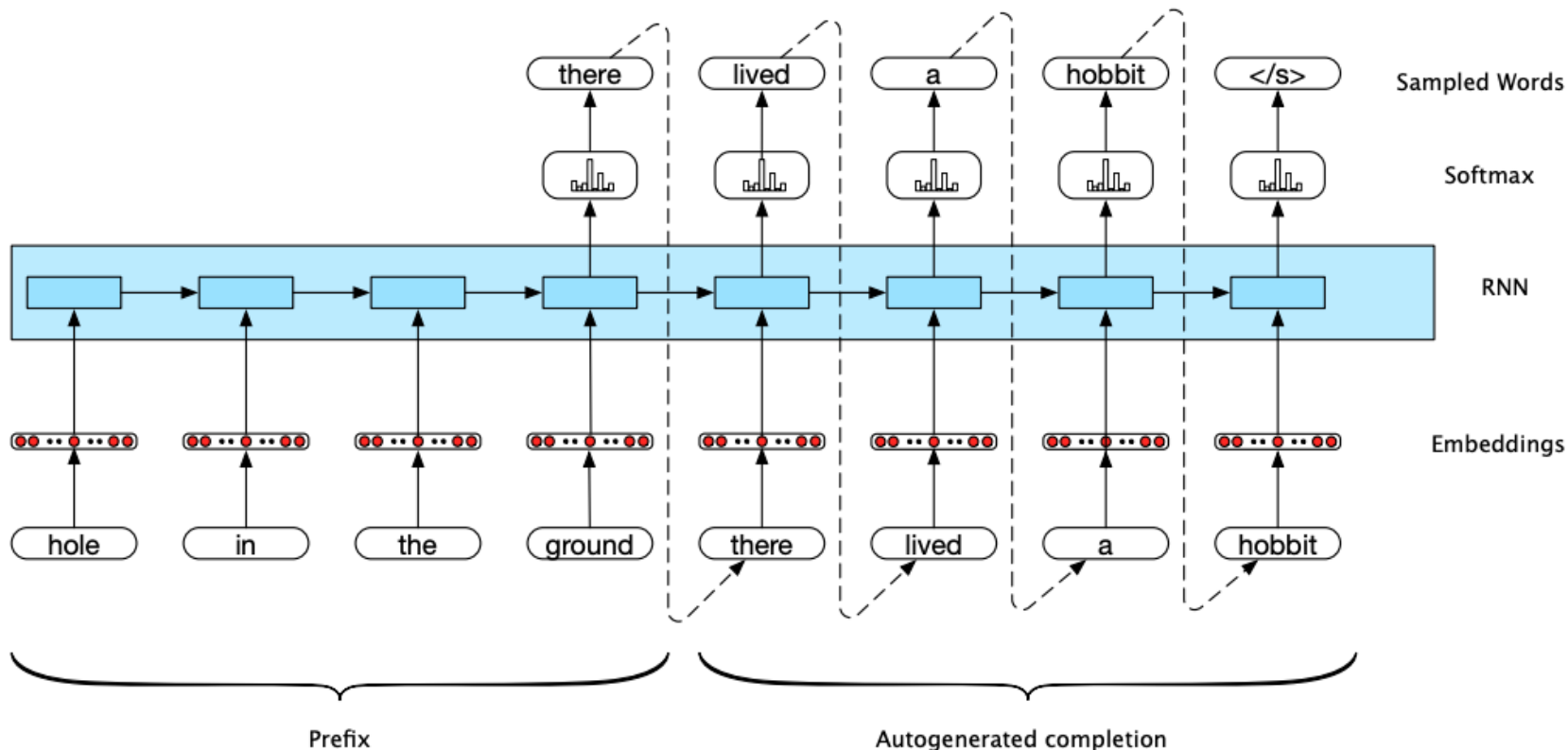
Autoregressive Generation

- We can use a trained RNN LM to generate novel sentences using the same Shannon method we used with N -gram language models (chapter 3).

Autoregressive Generation (Decoding)



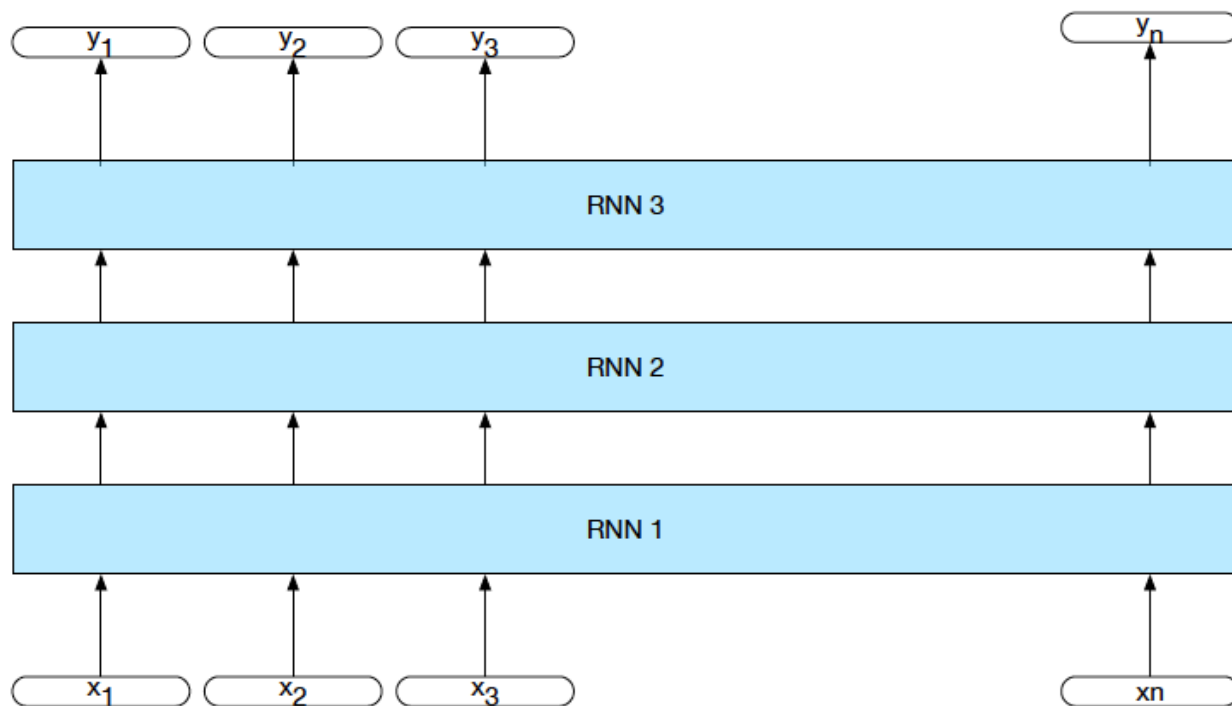
Autoregressive Completion



Deep Learning

- We can view these networks as black boxes with vectors going in and out.
- This modular view leads to more and more complex architectures.
 - Stacking layers

Stacked RNNs



Problem

The performance of recurrent networks degrades as sequences get longer.

- Hard to train successfully
- Hard to get the hidden units to remember (and forget) exactly what they need to remember over time.

LM Example

The flights the airline was cancelling were full.

LM Example

The flights the airline was cancelling were full.

LM Example

The flights the airline was cancelling were full.

English has "number agreement" between verbs and their subjects.

- "Flights" is plural, so the verb form is "were" (plural)
- "airline is singular" so the verb form is "was" (singular)

But if we're using an RNN to process this sentence what does it recall about flights once it gets to "cancelling"?

The probability assigned by the LM to "were" should be higher than "was"

Attention

- Attention by itself is a powerful approach
 - Without the need for recurrence

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

Transformers

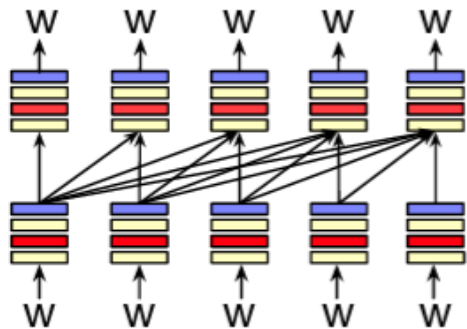
Three architectures

1. Causal autoregressive attention models
2. Encoder-decoder models with attention
3. Bidirectional encoder models

Architectures

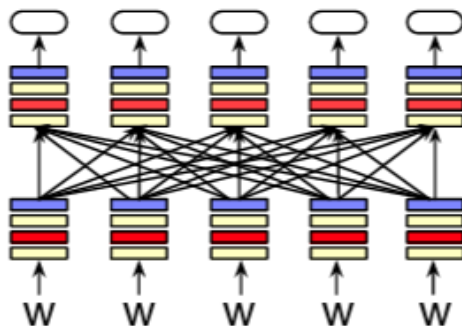
1. Language Models
2. Gradient-based learning
3. Embeddings

GPT family



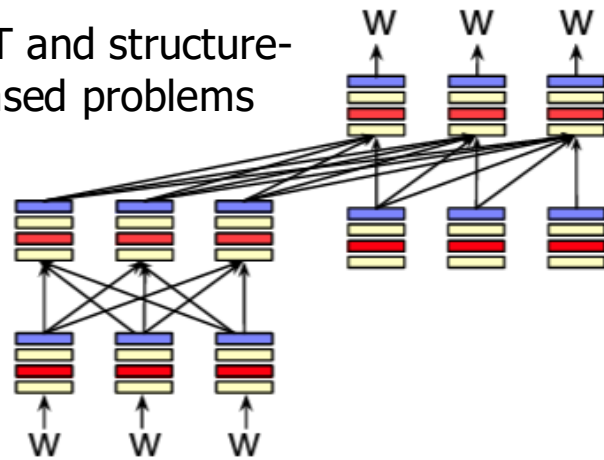
Decoder

BERT family



Encoder

MT and structure-
Based problems



Encoder-Decoder

Attention Mechanism

- Attention, like simple NNs, is another “biologically inspired” technique borrowed from cognitive science. How do you allocate attentional resources to focus on a small portion of what’s important in a larger input field.
 - Visual attention
 - Picking out the part of scene that matters to you at the moment
 - Cocktail party effect
 - Tracking a single conversation in a stream of conversational noise.

Language-Based Attention

- Differentially attend to earlier items in a text based on the current focus of attention...
 - *Based on the horrific car accident he became an organ ____*
 - *Based on his extensive training and practice he became an organ__*
- How do you interpret organ based on the earlier context?

JA

Finish this story: Based on the horrific car accident he became an organ



Based on the horrific car accident, he became an organ donor. In the midst of tragedy, there was a glimmer of hope that his passing might bring a second chance at life to someone else.

News of his selfless decision spread quickly through the hospital corridors, touching the hearts of the medical staff. It wasn't long before his organs were carefully harvested, each one destined to find a new home in a body that desperately needed it.

As the days passed, families sat in the waiting rooms, hoping for a miracle. Among them was Lily, a young woman with a failing heart. Her days were spent tethered to machines, her once vibrant spirit diminished by the relentless toll of her illness. Time was running out.

JA

Finish this story: Based on his extensive training and practice he became an organ



Based on his extensive training and practice, he became an organ virtuoso, renowned for his ability to breathe life into the keys of a grand piano. The world had never witnessed such mastery, each note he played seemed to carry a piece of his soul.

As his reputation grew, concert halls across the globe clamored for his performances. People traveled from far and wide to witness the magic he conjured with his fingers. His music transcended boundaries, speaking a universal language that resonated with the hearts of all who listened.

Word Embeddings

- In most approaches, we have 1 word embedding per token in the vocabulary.
- Regardless of how many distinct senses a word has.
- So how can that embedding capture the alternative meanings?

Contextual Embeddings

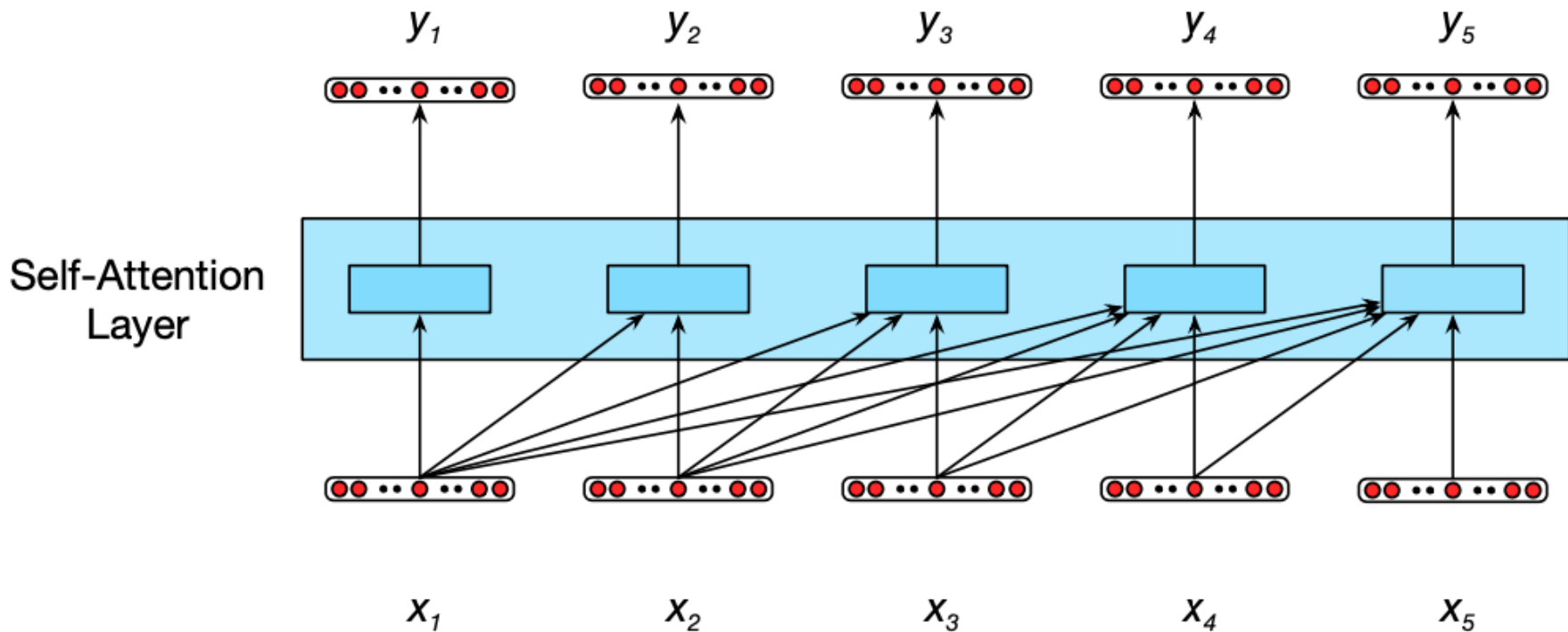
- Words (vectors) in context must incorporate information from the discourse context to adjust the vectors to represent the intended meaning.
- This is where simple vector arithmetic comes in useful.

Self-Attention:

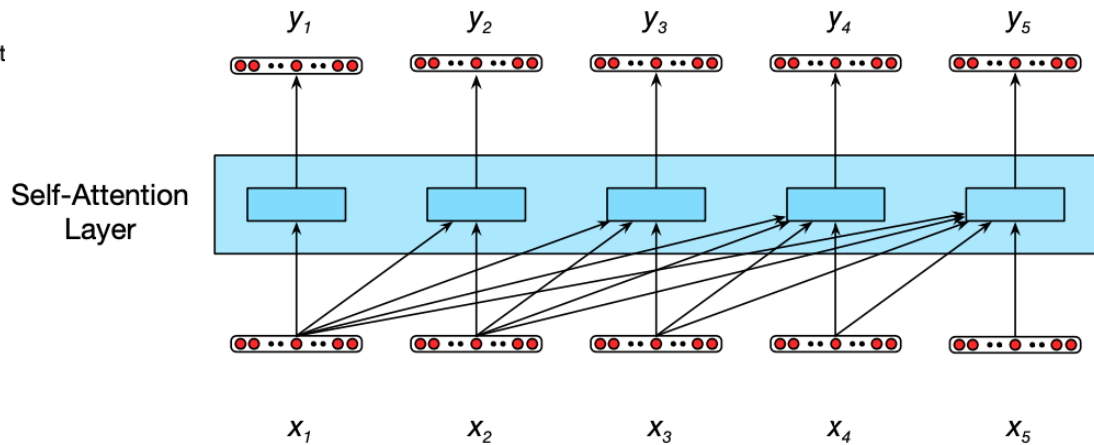
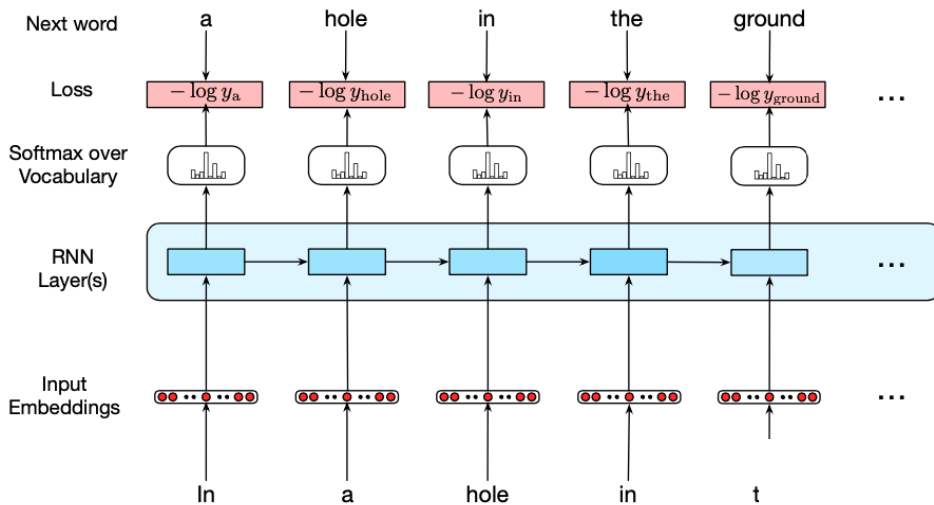
Causal Sequence Processing

- Causal here just means left to right processing
 - *What happens next is caused by what happened in the past.*
- Compare the current token to each previous item in a sequence and generate a similarity score to each
 - Normalize those scores to produce a probability distribution
- Use the normalized scores to produce an output based on the weighted sum of the preceding inputs.

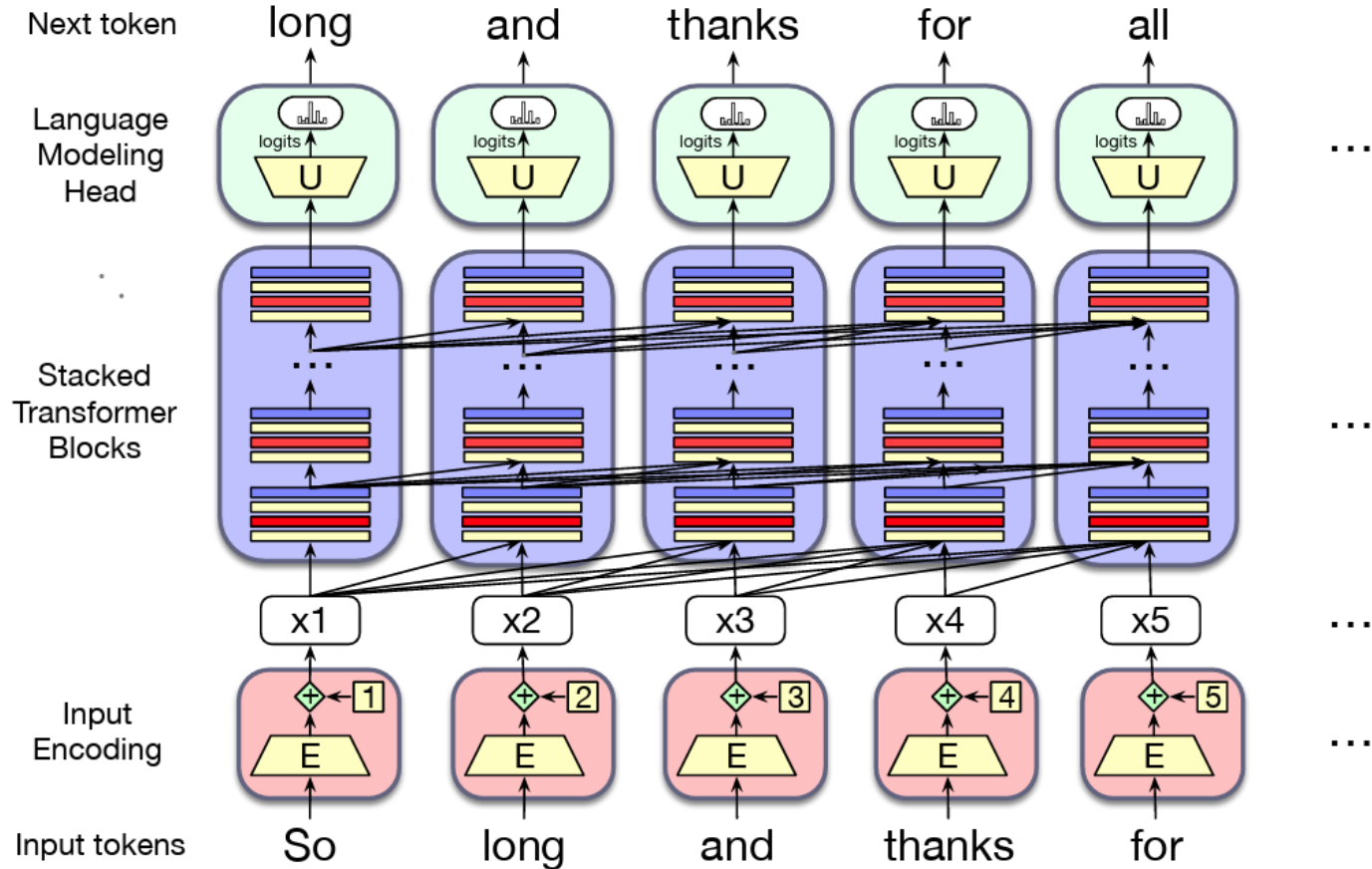
Causal Transformers (GPT*, etc).



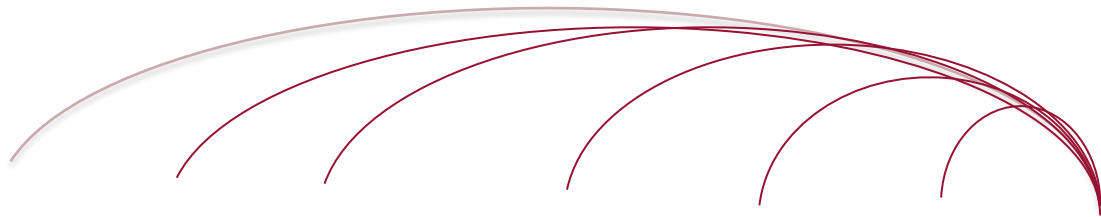
Compared to RNNs



Causal Transformers



Simple Attention in Sequence Processing



...horrific car accident he became an organ

$$\text{score}(x_i, x_j) = x_i \cdot x_j$$

$$\begin{aligned}\alpha_{ij} &= \text{softmax}(\text{score}(x_i, x_j)) \quad \forall j \leq i \\ &= \frac{\exp(\text{score}(x_i, x_j))}{\sum_{k=1}^i \exp(\text{score}(x_i, x_k))} \quad \forall j \leq i\end{aligned}$$

$$y_i = \sum_{j \leq i} \alpha_{ij} x_j$$

Example

Assume x is the vector of embeddings for “accident, he, became, an, organ”

The focus is “organ”. Take the dot product between the focus embedding and all the other embeddings with the following vector as the result.

<u>Focus</u>	Accident	He	Became	An	Organ
organ	10	2	5	7	25

Softmax produces α_6

[3.05902222e-07, 1.02618763e-10, 2.06115296e-09, 1.52299748e-08, 9.99999677e-01]

$$y_6 = \sum_{j \leq 6} \alpha_{6,j} \times x_j$$